



# Docker

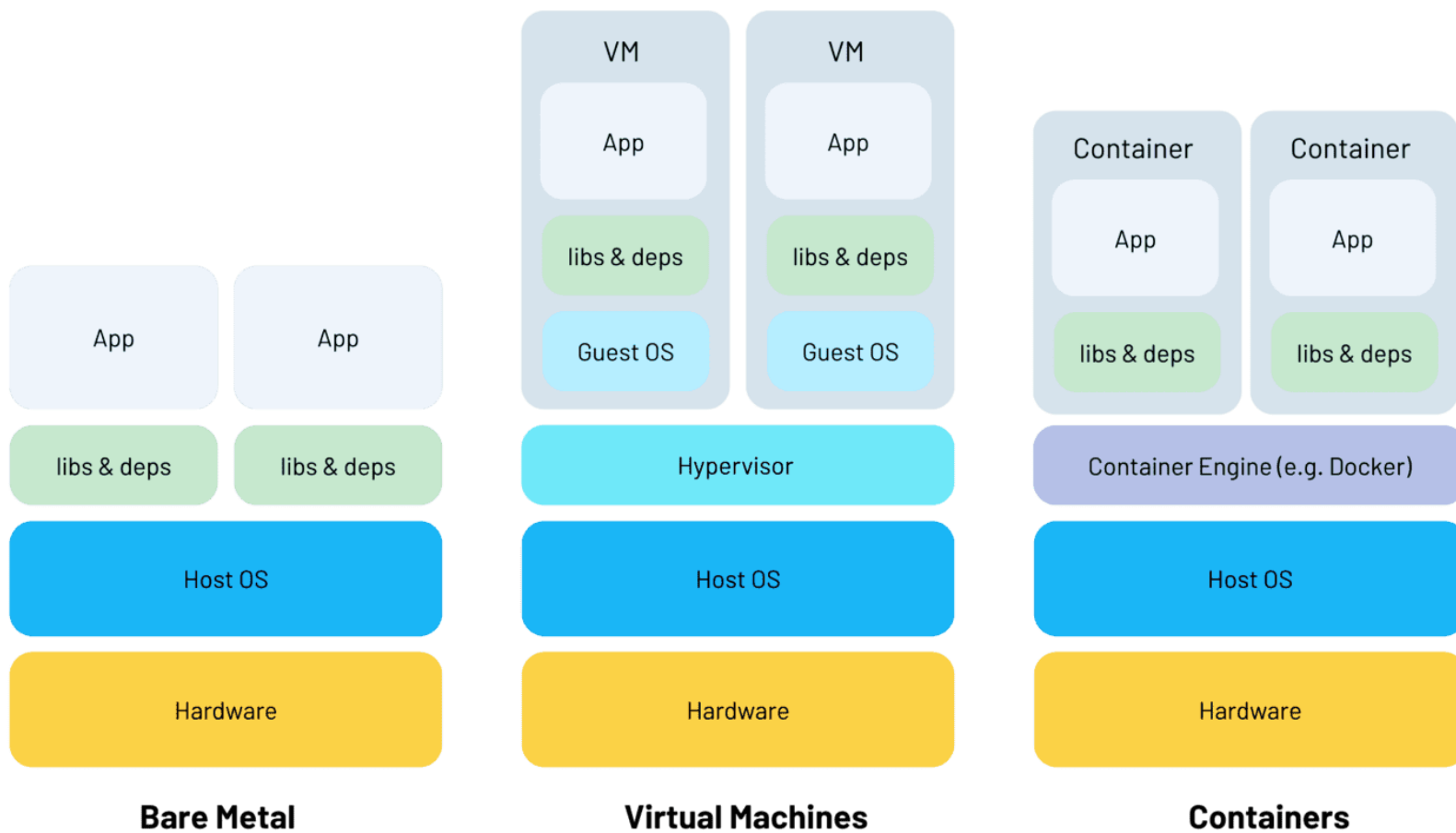
**Containers - dive in head first**

**Erfan Taheri**

## What is a container?

- ▶ **A lightweight, portable runtime for an application and its dependencies Executes as isolated processes on the host kernel (not a full VM) Built from layered images and instantiated as containers**Containers are:
  - ▶ **Self-contained.** Each container has everything it needs to function with no reliance on any pre-installed dependencies on the host machine.
  - ▶ **Isolated.** Since containers are run in isolation, they have minimal influence on the host and other containers, increasing the security of your applications.
  - ▶ **Independent.** Each container is independently managed. Deleting one container won't affect any others.
  - ▶ **Portable.** Containers can run anywhere! The container that runs on your development machine will work the same way in a data center or anywhere in the cloud!

# Bare Metal vs Virtual Machines vs Containers



## Why Containerization is becoming a major trend?

- ▶ **Agility**
- ▶ **Micro-service application deployment architecture**
- ▶ **Resource Management (Efficiency)**
- ▶ **Operation & security overhead (Ease of management)**
- ▶ **Eliminating build and implantation overheads**
- ▶ **Test and operation environments scenarios**
- ▶ **Orchestration and automation**
- ▶ **Scalability**
- ▶ **Portability**

## Benefits of using containers

| Benefit              | Concrete Example                                                                             |
|----------------------|----------------------------------------------------------------------------------------------|
| Portability          | Develop on laptop (same image) → push to registry → run on cloud node without config drift   |
| Build                | CI builds image with exact deps; QA & Prod run same image → fewer 'works on my machine' bugs |
| Efficiency & Density | Run many lightweight containers per host vs fewer heavy VMs — better CPU/memory utilization  |
| Isolation & Security | Limit process view (PID ns) and resource use (cgroups) for multi-tenant hosts                |
| Fast scaling         | Auto-scale stateless containers under load; start in seconds rather than minutes             |

## Disadvantages

| Disadvantage              | Explanation                                                                      | Example / Impact                                               |
|---------------------------|----------------------------------------------------------------------------------|----------------------------------------------------------------|
| Weaker isolation than VMs | Containers share the host kernel, so a kernel exploit may affect all containers. | A malicious container escape could compromise the entire host. |
| Security complexity       | Requires extra measures like SELinux/AppArmor, seccomp, rootless containers.     | More hardening steps compared to VMs.                          |
| Networking complexity     | Overlay networks, port mapping, and DNS management add complexity.               | troubleshoot networking is harder than in traditional apps.    |

# Foundational mechanisms for containers

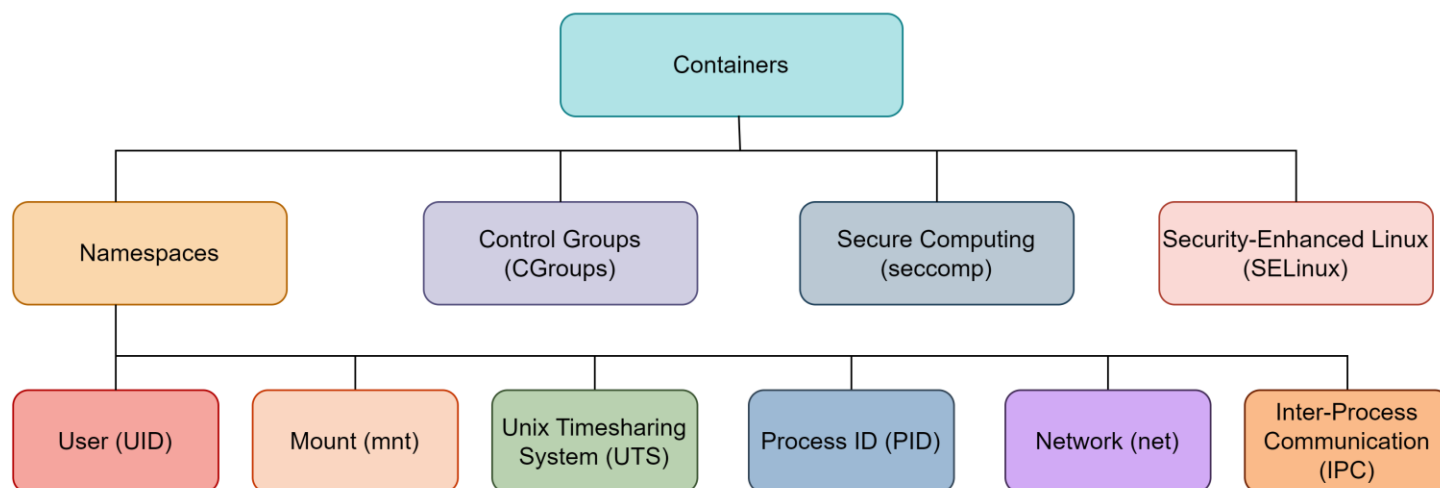
## ▶ Namespaces

▶ isolate kernel resources (pid, net, mnt, ipc, uts, user)

## ▶ Cgroups

▶ control and enforce resource limits (cpu, memory, blkio, pids)

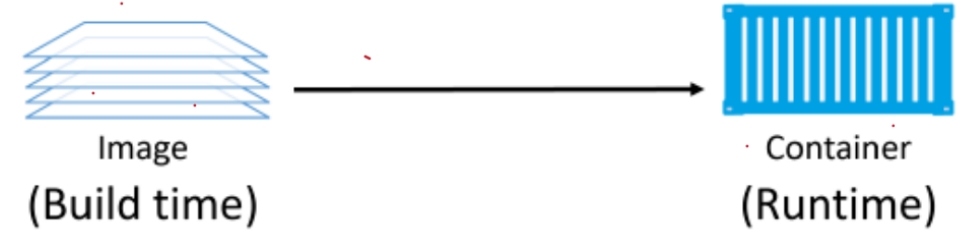
layered images (overlayfs) provide copy-on-write and small images



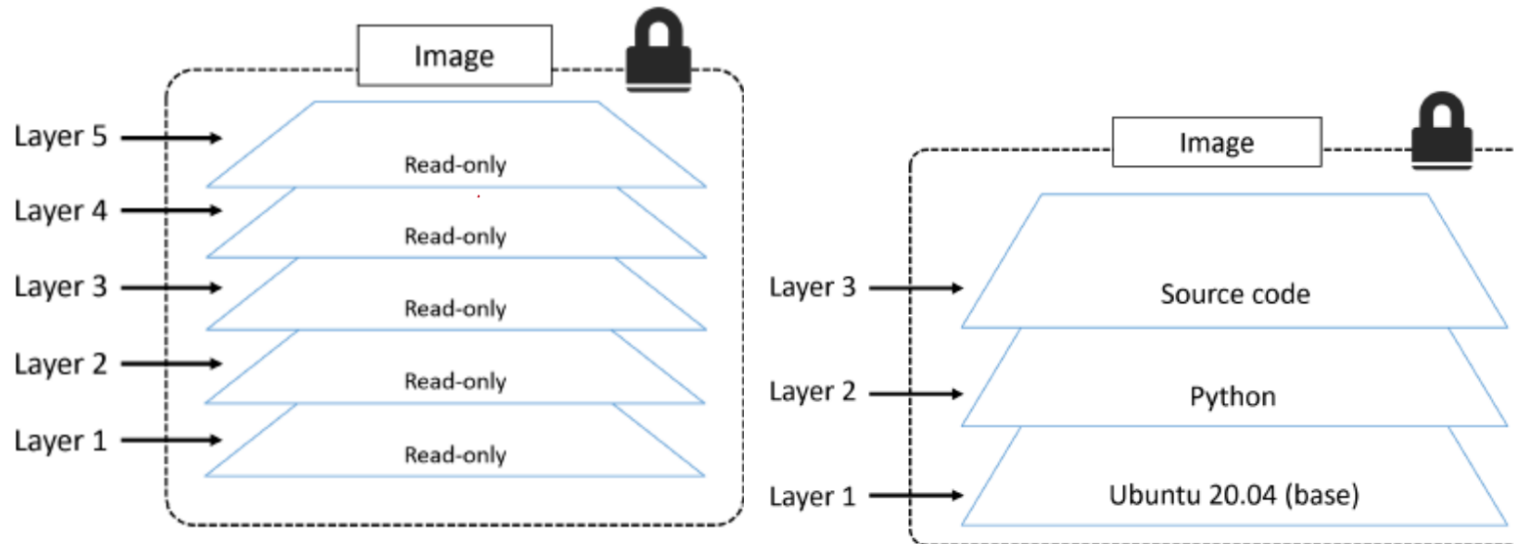
## Namespace Isolation - Quick Reference

| Namespace | Isolates         | Notes                                                               |
|-----------|------------------|---------------------------------------------------------------------|
| pid       | Process IDs      | PID 1 semantics differ; use tini/s6 or write proper signal handlers |
| net       | Networking stack | veth pairs, bridge, iptables/nft NAT; macvlan/ipvlan options        |
| mnt       | Mount points     | Private mount namespace; overlayfs provides merged view             |
| ipc       | Shared memory    | Isolates SysV and POSIX IPC objects                                 |
| uts       | Hostname/domain  | Per-container hostname                                              |
| user      | UID/GID mappings | Rootless containers map user IDs; limits kernel ABI access          |





- ▶ A content-addressable bundle of filesystem layers + metadata
- ▶ Identified by digests (sha256:<hash>), not just tags
- ▶ Manifest describes layers; Image Index (Manifest List) maps to architectures



# Image Tag, Hash and Layer

## ► Container Image

- A **portable, read-only package** that contains everything needed to run a containerized application.
- Includes: base OS (optional), runtime libraries, dependencies, and the application code.
- Built in layers for efficiency and portability.

## ► Tags

- A **human-friendly label** to identify an image version.
- Example: myapp:1.0 vs. myapp:2.0.
- The tag latest often points to the newest version, but may change over time.
- Useful for versioning and deployment workflows.

## Image Tag, Hash and Layer (cont'd)

### ► Layers

- Each image is composed of **multiple immutable layers** stacked together.
- Layers are reused across images to reduce storage and speed up pulls/builds.
- Example structure:
  - Base layer → Operating system or runtime
  - Dependency layer → Libraries, frameworks
  - Application layer → App binaries, configs

### ► Image Hash (Digest)

- A **checksum (SHA256)** uniquely identifying an image/image layer.
- Guarantees immutability: the digest never changes, unlike tags.
- Example: `myapp@sha256:3d2f4e1b...`
- Critical for security and reproducibility in deployments.

# Image Layers & Manifest

Manifest (schema2)

- mediaType: application/vnd.oci.image.manifest.v1+json
- config + layers[] digests



Layer 4: App code (sha256:...f4)

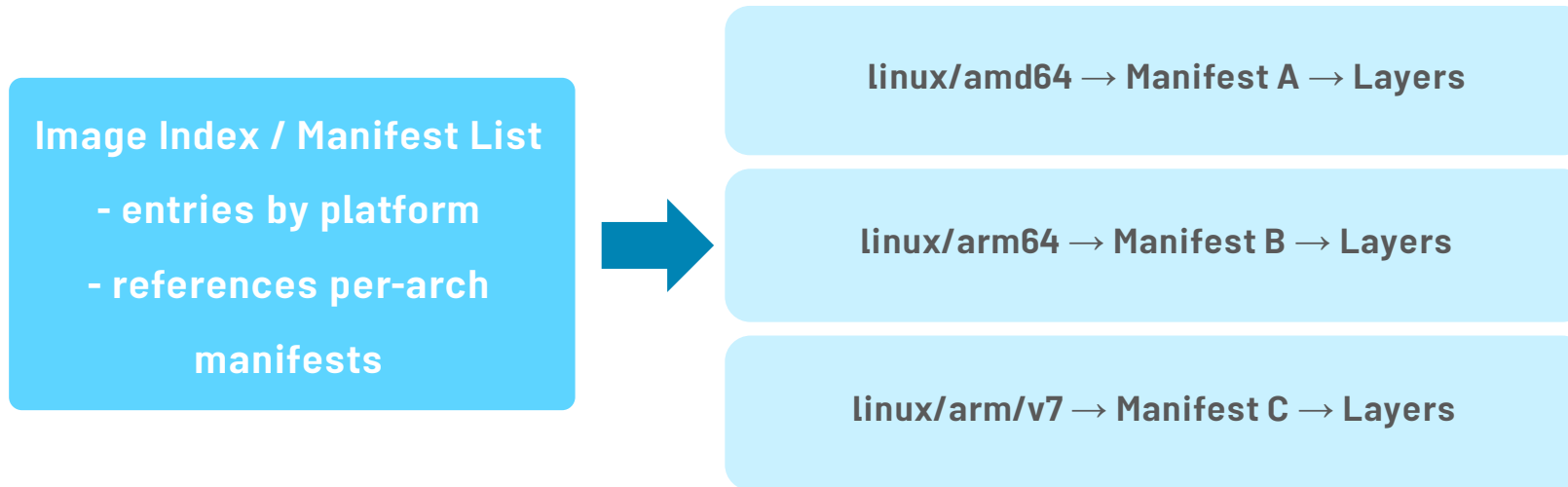
Layer 3: App deps (sha256:...a3)

Layer 2: OS packages (sha256:...b2)

Layer 1: Base (scratch/alpine) (sha256:...c1)

## Image Index (Multi-Arch)

- ▶ The manifest listpoints to specific image manifests for one or more platforms. Its use is optional.



- ▶ The image manifest provides a configuration and a set of layers for a container image

# Container Registries

## ▶ What is a Container Registry?

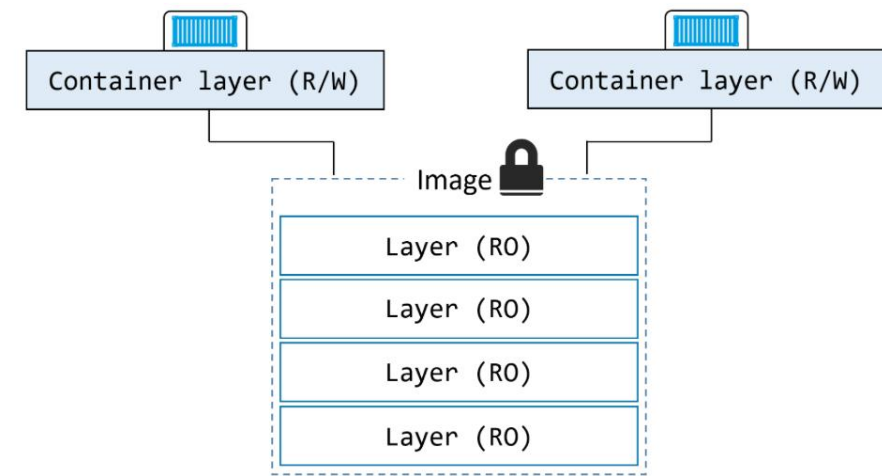
- ▶ A **storage and distribution system** for container images.
- ▶ Functions like a **package repository** but for images.
- ▶ Supports:
  - ▶ **Push** (upload images)
  - ▶ **Pull** (download images)
- ▶ Can be **public** (shared with everyone) or **private** (restricted access).

# Ephemeral Storage

**Ephemeral writable storage** When a container starts, Docker adds a thin, writable layer on top of the image layers.

## ► Implements Copy-on-Write (CoW):

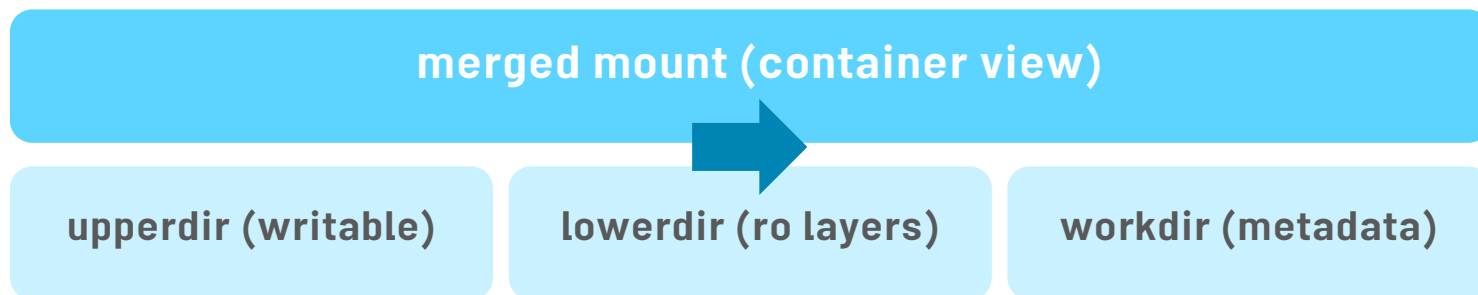
- If a process modifies a file, Docker copies it from the read-only base layer into the writable layer, and modifies it there.
- If no modification occurs, the container still reads from the base layer.
- Ensures that multiple containers can share the same base image without interfering with each other.
- located:
  - Linux Docker hosts: `/var/lib/docker/<storage-driver>/...`
  - Windows Docker hosts: `C:\ProgramData\Docker\windowsfilter\...`



## overlayfs (lower/upper/work → merged)

### ► What is OverlayFS?

- A **union filesystem** used by containers to combine multiple layers into one.
- Provides **copy-on-write**: only changes are written on top, saving space and time.



- Merged view: upperdir (writable), lowerdir (ro layers), workdir (metadata)
- Copy-on-write: first write copies a file from lower to upper
- Implications: inode changes on COW; performance varies with small-file churn
- Debug mounts with: `mount | grep overlay`; inspect `/var/lib/docker/overlay2`



# overlays (lower/upper/work → merged)

## ▶ Key Components

### ▶ Lower Directory (Read-Only Layers)

- ▶ Contains **the base image layers** (OS, libraries, dependencies).
- ▶ Immutable: cannot be changed.
- ▶ Multiple lower layers can be stacked.

### ▶ Upper Directory (Writable Layer)

- ▶ Stores **changes made at runtime** (new files, modifications, deletes).
- ▶ Exclusive to the running container.

### ▶ Work Directory

- ▶ A **scratch space** required by OverlayFS to manage operations (copy, rename, delete).
- ▶ Internal use only — not directly visible to the container.

### ▶ Merged Directory

- ▶ The **final unified view** presented to the container.
- ▶ Combines lower (read-only) + upper (writable) → looks like a single filesystem.

# Container Lifecycle

▶ create → start → running → stopped → removed

▶ Ephemeral Storage Lifecycle tied to container lifecycle :

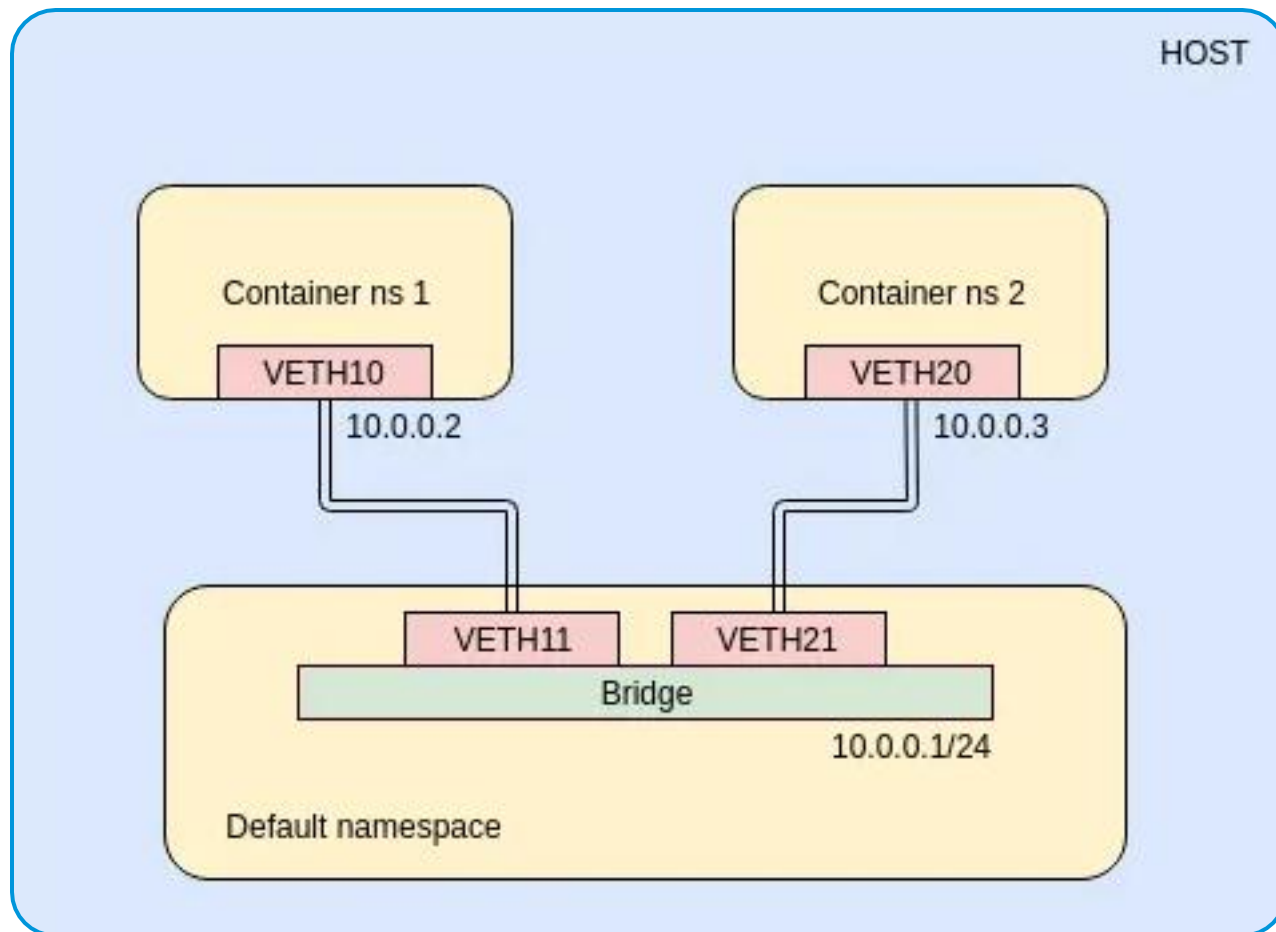
▶ **Create/Start:** Writable (ephemeral) layer is allocated

▶ **Running:** All runtime changes (logs, temp files, downloads) stored in ephemeral layer

▶ **Stopped:** Data still exists on disk, but tied only to that container

▶ **Removed:** Ephemeral storage is deleted → all changes are lost

## Container Networking - veth + bridge

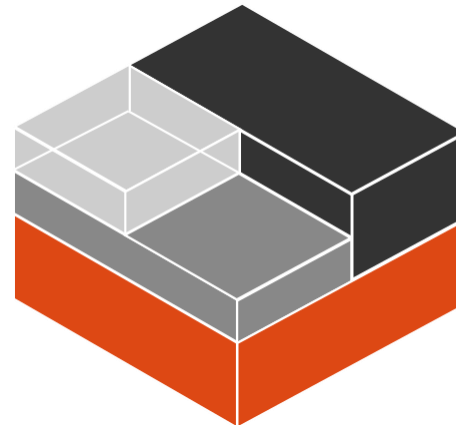


## LXC (Linux Containers)

- ▶ LXC emerged around 2008: user-space tooling around kernel namespaces & cgroups Provided an early, complete container manager using only upstream Linux kernel features.

### ▶ Key Technologies Used

- ▶ Namespaces → isolate processes, network, users, IPC.
- ▶ cgroups → limit and account CPU, memory, I/O.
- ▶ Union filesystems → efficient image layering.



# Docker & Docker, Inc.

## ► Origins

- Started at dotCloud, a PaaS company (2010s).
- Created by Solomon Hykes to simplify software packaging.

## ► Key Milestones

- 2013 → Docker open-sourced at PyCon.
- 2014 → Docker Hub launched (public image registry).
- 2015 → Docker became industry standard, ecosystem grew (Compose, Swarm).
- 2016–2017 → Split core into containerd (runtime) and contributed to CNCF.
- 2017–2019 → Rise of Kubernetes shifted Docker's role (dockershim, containerd adoption).
- 2020+ → Docker refocused on developer tools (Docker Desktop, BuildKit).

# Docker Architecture

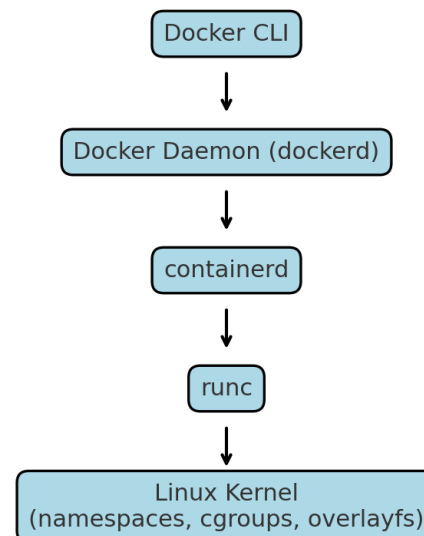
## ► Architecture Overview

- Docker CLI → User command interface (docker run, docker build).
- Docker Daemon (dockerd) → Manages containers, images, networks, volumes.
- containerd → High-level runtime for container lifecycle management.
- runc → Low-level runtime; creates containers using Linux namespaces & cgroups.
- Kernel features → Namespaces, cgroups, overlayfs.

## ► Strengths

- Modular design (CLI → dockerd → containerd → runc).
- Broad ecosystem & integration (Docker Hub, Compose, Swarm, Kubernetes).
- Developer-friendly tooling (simple CLI, Desktop app).
- Strong image distribution workflow (layered filesystem).

### Docker Architecture (Simplified)



## Docker Architecture (cont'd)

### Historically Monolithic

- ▶ In its early years (2013–2015), Docker Engine (dockerd) was a single, monolithic daemon that handled everything:
  - ▶ Image building and management
  - ▶ Networking setup
  - ▶ Container lifecycle (create, run, stop)
  - ▶ Low-level interaction with Linux kernel (namespaces, cgroups)
- ▶ This made Docker simple for developers, but also:
  - ▶ Created a large attack surface (security risk).
  - ▶ Made it harder to modularize and integrate with other tools.
  - ▶ Limited adoption in ecosystems like Kubernetes that wanted pluggable runtimes.

## Evolution: OCI, containerd, Moby

- ▶ **Open Container Initiative (OCI) formed June 2015 to create vendor-neutral specs (runtime + image)**
- ▶ **Docker separated components over time: containerd (runtime daemon), runc (OCI runtime), Moby as upstream toolkit**
- ▶ **containerd handles lifecycle and image transfer; runc handles low-level namespace/cgroup setup**



## further readings

| Topic                                                  | Source                                                                                                             |
|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| History of LXC / Linux containers<br>(Red Hat summary) | The History of Containers – Red Hat ( <a href="#">Red Hat</a> )                                                    |
| History of LXC / Linux containers<br>(Oracle summary)  | A Brief History of Linux Containers – Oracle<br>( <a href="#">blogs.oracle.com</a> )                               |
| Docker history & public demo at<br>PyCon 2013          | 11 Years of Docker: Shaping the Next Decade<br>– Docker blog ( <a href="#">Docker</a> )                            |
| Namespaces & cgroups<br>explanation                    | What Are Namespaces and cgroups... –<br>NGINX blog ( <a href="#">blog.nginx.org</a> )                              |
| Docker architecture & containerd<br>role               | containerd vs. Docker – Docker blog ( <a href="#">Docker</a> )                                                     |
| OCI formation & specifications                         | About the Open Container Initiative –<br><a href="#">opencontainers.org</a> ( <a href="#">opencontainers.org</a> ) |